NYC Urban Mobility Explorer: Report

1. Problem Framing and Dataset Analysis

Dataset. We used the NYC Taxi & Limousine Commission's January 2019 yellow taxi records: 7,667,792 trips (a 648 MB CSV) with times, zones, distance, passengers, payment type and fares, plus a zone lookup and a shapefile. We clean them, store them in a normalised database, and explore them in a dashboard.

Challenges and cleaning. There were no duplicates or unknown zones, but many records had impossible values. We kept 7,555,473 trips (98.54%) and excluded 112,319 (1.46%). The main exclusion rules were trips under 1 minute (67,755), trips over 4 hours (20,694), zero distance (11,436) and dropoff not after pickup (6,291). *Assumptions:* a real trip happens in January, lasts 1 minute to 4 hours, covers at most 100 mi at no more than 80 mph, and carries at least one passenger. Cash tips are unknown, not zero.

Unexpected observation. 114,741 trips recorded 0 passengers, more than all the excluded records combined, yet their times and fares were normal. Deleting them would have removed real trips. So we gave the logger three outcomes: **excluded**, **fixed** (passenger count set to 1) and **suspicious** (310 kept with a flag). All three are written to `logs/excluding_records.csv`.

Derived features. `trip_duration_min`, `avg_speed_mph`, `fare_per_mile`, `tip_pct` (card trips only), time-of-day and weekday fields, `is_airport_trip` and `is_cross_borough`.

2. System Architecture and Design Decisions

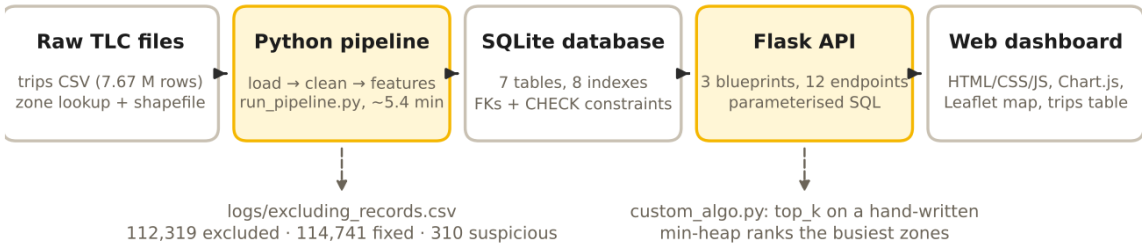


Figure 1: Architecture. One command rebuilds the database in about 5.4 minutes.

Stack and schema. pandas handles 7.6 M rows in memory. SQLite needs no server, so each teammate can rebuild the same database. Flask keeps the backend in Python, and the frontend uses plain JS with Chart.js and Leaflet. The database has seven tables: the `trips` fact table, five lookup tables (vendors, rate codes, payment types, boroughs, zones) and `zone_shapes` (GeoJSON). Eight foreign keys, CHECK constraints and eight indexes back the filters. All queries use `?` placeholders, and sort columns come from a whitelist.

Trade-offs. SQLite is simple but allows one writer at a time and has no spatial queries. Storing the derived features takes extra space, but indexed filters are faster than computing values per row. Simplified polygons lose some boundary detail, but the map loads much faster.

3. Algorithmic Logic and Data Structures

The busiest-zones ranking uses our own `top_k` (`custom_algo.py`), built on a hand-written binary min-heap stored in a list (parent at (i − 1) // 2, children at 2i + 1 and 2i + 2). It uses no `ORDER BY`, `sorted`, `heapq` or `Counter`. SQL only counts the pickups per zone. The heap holds at most k zones, with the weakest at the top, and any new zone that beats the weakest replaces it.

```
top_k(items, k, key):
    heap = empty min-heap
    for item in items:
        if size(heap) < k: push(heap, item)          # sift up
        else if key(item) > key(heap[0]):
            replace_top(heap, item)                  # sift down
    return pop all items, reversed                    # biggest first
```

Complexity: O(n log k) time and O(k) memory, against O(n log n) for a full sort. A self-test matches `sorted` for k = 1 to 600 on 500 random items.

4. Insights and Interpretation

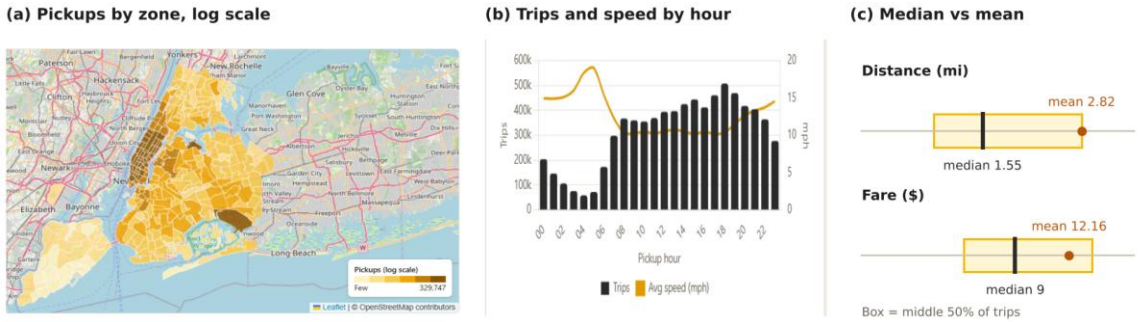


Figure 2: Full-month dashboard views of 7,555,473 trips.

4.1 Demand is concentrated in Manhattan's core (Fig. 2a; `/zone-activity` and `/top-zones`). All 10 of the busiest zones are in Manhattan, led by Upper East Side South with 329,747 pickups. The airports stand out on the map but are only 6.4% of trips. *Meaning:* yellow cabs serve the places where street hails work, so taxi and congestion planning should focus on a few Manhattan zones.

4.2 Most trips are short hops (Fig. 2c; full-month summary). The median trip is 1.55 mi and $9.00, and 75% of trips are under 2.82 mi. The means (2.82 mi, $12.16) are pulled up by a few long airport runs. *Meaning:* taxis mostly replace a short walk or subway ride, and the median describes a typical trip better than the mean. The same skew is why the map uses a log scale.

4.3 Busy hours are slow hours (Fig. 2b; `/hourly`). The peak is 18:00 (507,911 trips). Speed is lowest at 17:00 (10.14 mph) and highest at 05:00 (18.99 mph). *Meaning:* cabs complete fewer trips per hour just when demand peaks, which is the problem the February 2019 congestion surcharge was designed to address.

5. Reflection and Future Work

Challenges. GitHub rejected our 648 MB CSV, so we untracked the data and added `.gitignore` rules. Two competing schemas were merged into one normalised design. Repeated inserts doubled the trips

to 981,644, so `init_db()` now recreates the tables before each load. Three different database file names became one `DB_PATH` in `config.py`. After a merge conflict brought back old code, our rule became: pull first, test after merging, and own separate files. Our 4-day development sample ranked Penn Station first, so every conclusion here uses the full month.

Next steps for a real product. Summary tables and caching; filters that also drive the charts and the map; multiple months loaded from parquet; PostgreSQL with PostGIS; and an online deployment with tests, CI, authentication and monitoring.

Team contributions:

Name	Role
Griphen Mweene	member
Aristote Henry Ngabo	member
Precious Amarachi Azubuike	Team lead